Simulate a uniform 2D distribution of fossils

Menghan Fu

2026-03-14

```
library(tibble)
library(ggplot2)
```

1 Generate one set of 2D uniform random points

Region: from 0 to 1 on the x-axis, and 0 to 1 on the y-axis.

```
set.seed(123)
n1 <- 1000
```

Generate n random x-coordinates from a uniform distribution. Generate n random y-coordinates from a uniform distribution.

```
x <- runif(n1, min = 0, max = 1)
y <- runif(n1, min = 0, max = 1)
```

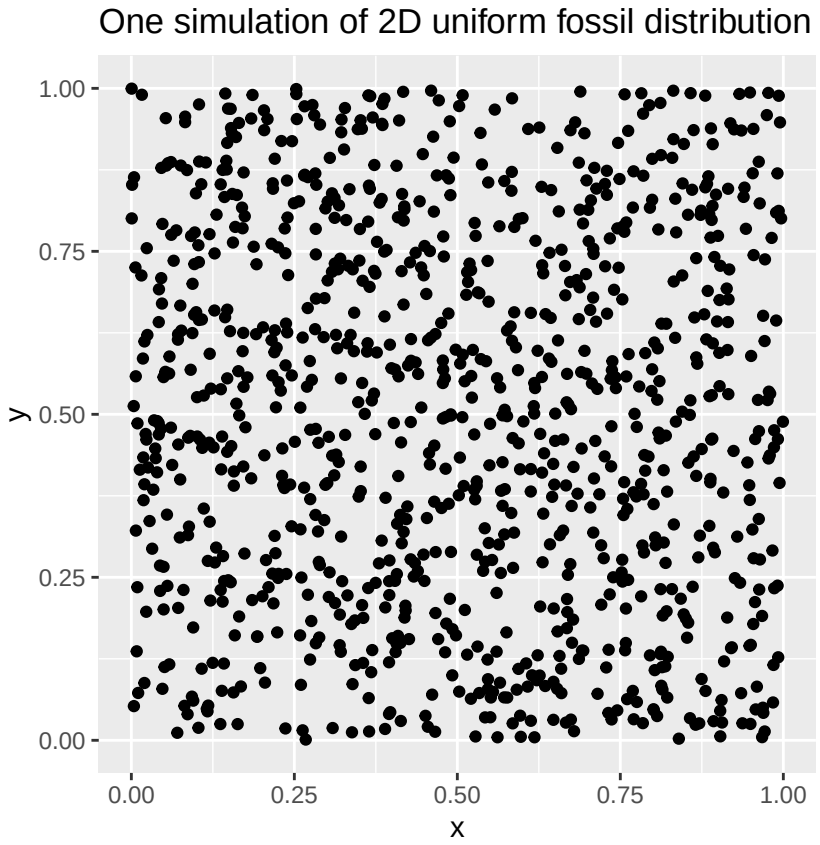
Combine x and y into an n x 2 tibble. Each row represents one point: (x, y)

```
points <- tibble(x = x, y = y)
head(points)
```

```
## # A tibble: 6 x 2
##        x     y
##    <dbl> <dbl>
## 1 0.288  0.274
## 2 0.788  0.594
## 3 0.409  0.160
## 4 0.883  0.853
## 5 0.940  0.848
## 6 0.0456 0.478
```

2 Plot the points using ggplot

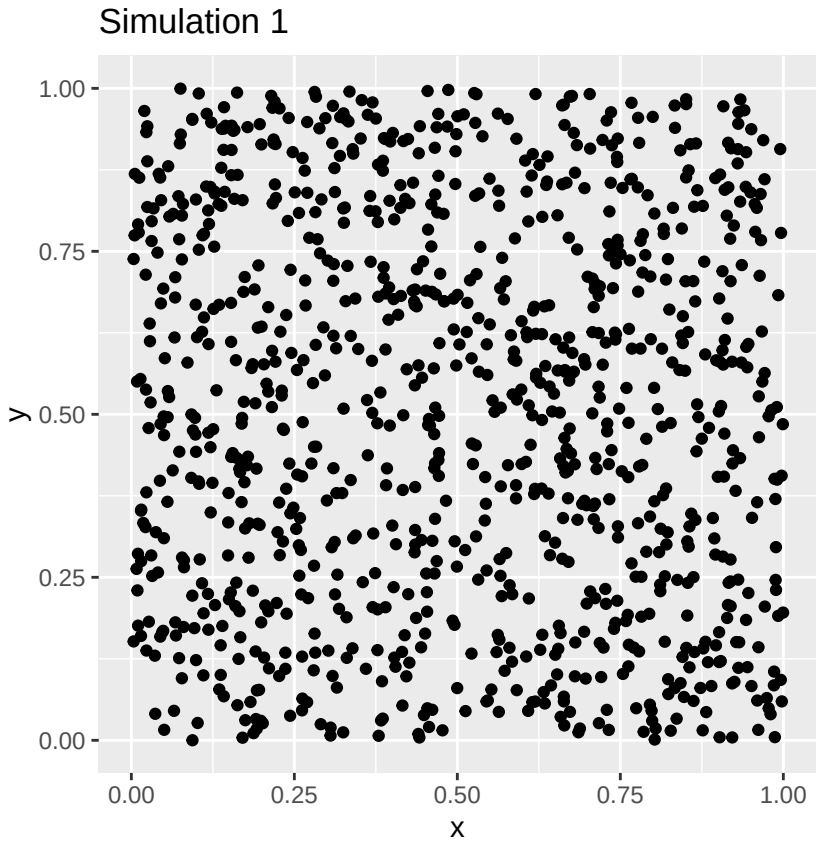
```
ggplot(points, aes(x = x, y = y)) +
  geom_point() +
  coord_fixed(xlim = c(0, 1), ylim = c(0, 1)) +
  ggtitle("One simulation of 2D uniform fossil distribution")
```

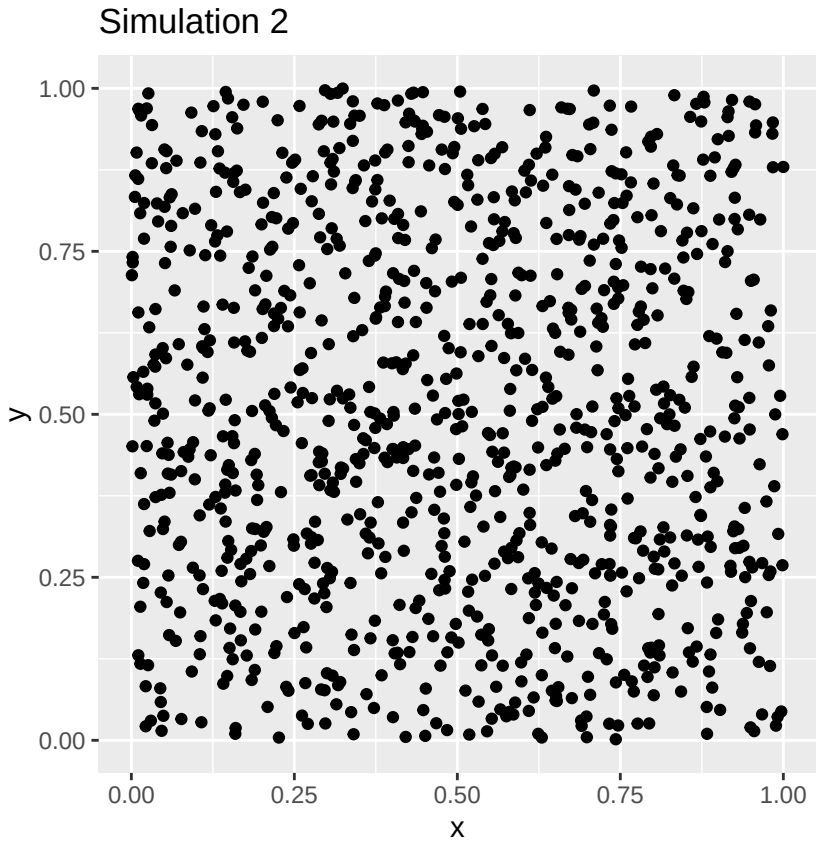


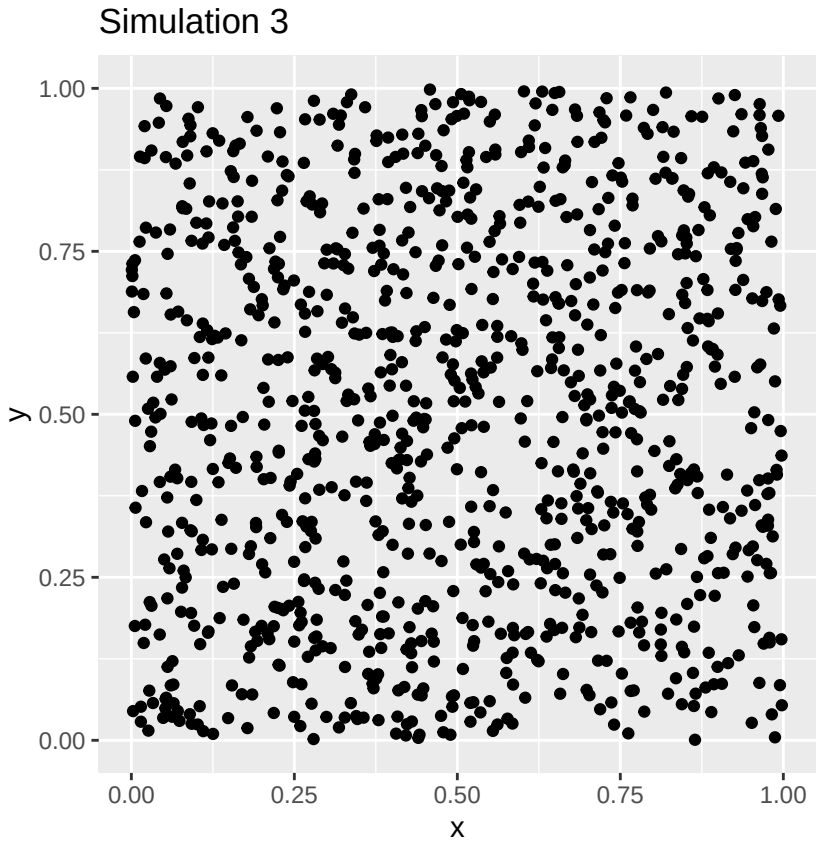
3 Use a loop to generate a series of simulations

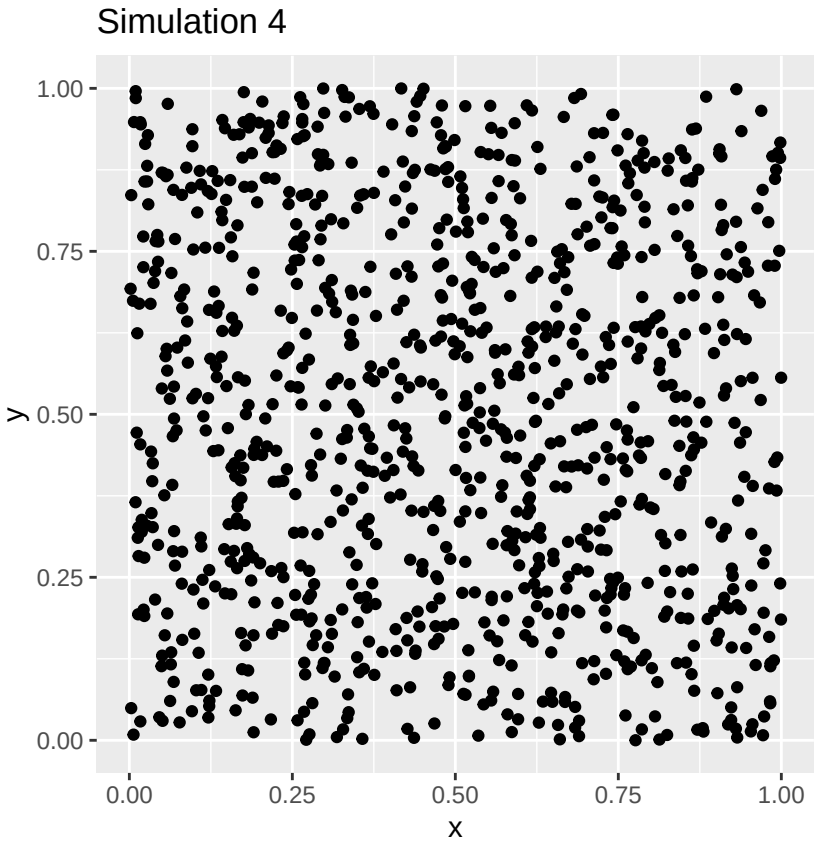
Here we do 5 simulations

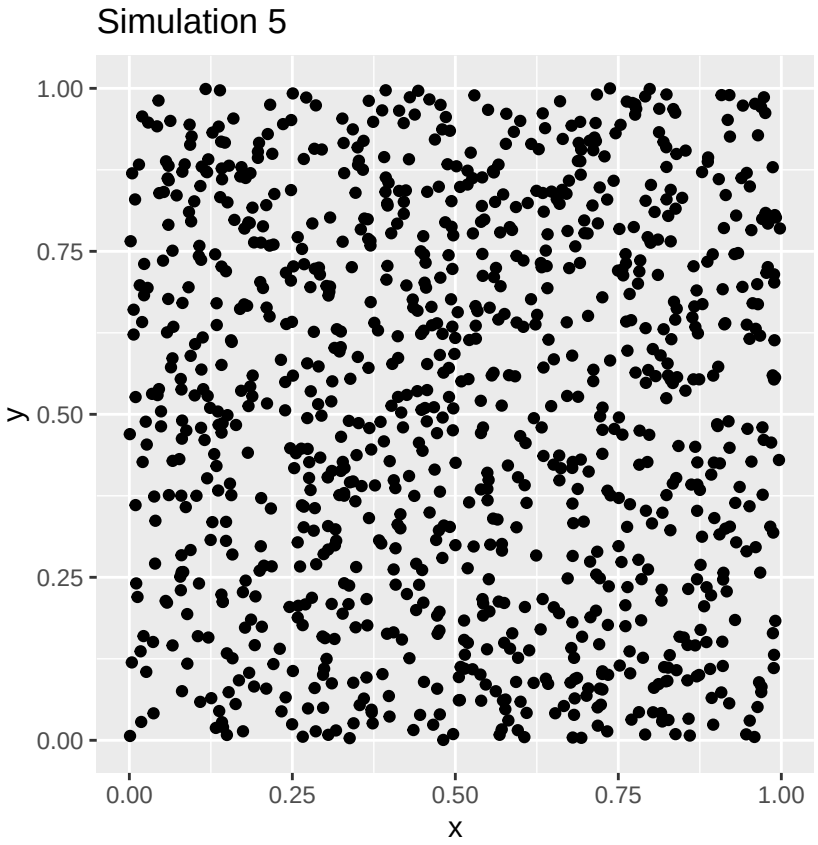
```
for (i in 1:5) {
  # Generate a new set of random x- and y-coordinates
  x <- runif(n1, min = 0, max = 1)
  y <- runif(n1, min = 0, max = 1)

  # Put the coordinates into a tibble
  points_s <- tibble(x = x, y = y)

  # Create a scatter plot for this simulation
  p <- ggplot(points_s, aes(x = x, y = y)) +
    geom_point() +
    coord_fixed(xlim = c(0, 1), ylim = c(0, 1)) +
    ggtitle(paste("Simulation", i))
  print(p)
}
```


Simulation 1
1.00
0.75
0.50
0.25
0.00
y
0.00
0.25
0.50
0.75
1.00
x


Simulation 2
1.00
0.75
0.50
0.25
0.00
y
0.00
0.25
0.50
0.75
1.00
x


Simulation 3
1.00
0.75
0.50
0.25
0.00
y
0.00
0.25
0.50
0.75
1.00
x


Simulation 4
1.00
0.75
0.50
0.25
0.00
y
0.00
0.25
0.50
0.75
1.00
x



4 Get the distances between the points

```
# Convert the tibble to a matrix
# Each row is still one point
point_matrix <- as.matrix(points)

# Compute all pairwise distances between the points
all_distances <- dist(point_matrix)

# Print all pairwise distances
as.matrix(all_distances)[1:10, 1:10]
```

```
##            1         2         3          4          5         6         7
## 1  0.0000000 0.5943774 0.1661505 0.83109891 0.86941055 0.3166986 0.5549082
## 2  0.5943774 0.0000000 0.5761684 0.27630332 0.29598045 0.7517492 0.3162925
## 3  0.1661505 0.5761684 0.0000000 0.83982355 0.86902991 0.4827100 0.6249663
## 4  0.8310989 0.2763033 0.8398236 0.00000000 0.05773108 0.9178091 0.3637590
## 5  0.8694106 0.2959804 0.8690299 0.05773108 0.00000000 0.9683264 0.4189573
## 6  0.3166986 0.7517492 0.4827100 0.91780915 0.96832643 0.0000000 0.5659985
## 7  0.5549082 0.3162925 0.6249663 0.36375903 0.41895729 0.5659985 0.0000000
## 8  0.6052334 0.3161047 0.5019955 0.55810935 0.55442501 0.8663011 0.6012384
## 9  0.3359800 0.5789160 0.1709833 0.85473918 0.87352378 0.6525874 0.7084482
## 10 0.2375556 0.3654173 0.2843669 0.59355140 0.63240022 0.4127519 0.3407431
```

```
##            8         9        10
## 1  0.6052334 0.3359800 0.2375556
## 2  0.3161047 0.5789160 0.3654173
## 3  0.5019955 0.1709833 0.2843669
## 4  0.5581094 0.8547392 0.5935514
## 5  0.5544250 0.8735238 0.6324002
## 6  0.8663011 0.6525874 0.4127519
## 7  0.6012384 0.7084482 0.3407431
## 8  0.0000000 0.4111755 0.4593354
## 9  0.4111755 0.0000000 0.3867100
## 10 0.4593354 0.3867100 0.0000000
```

5 Compare all inter-event distances with Diggle's H(t)

Diggle gives the theoretical distribution function H(t) for the distance between two random points in a unit square.

For a unit square:

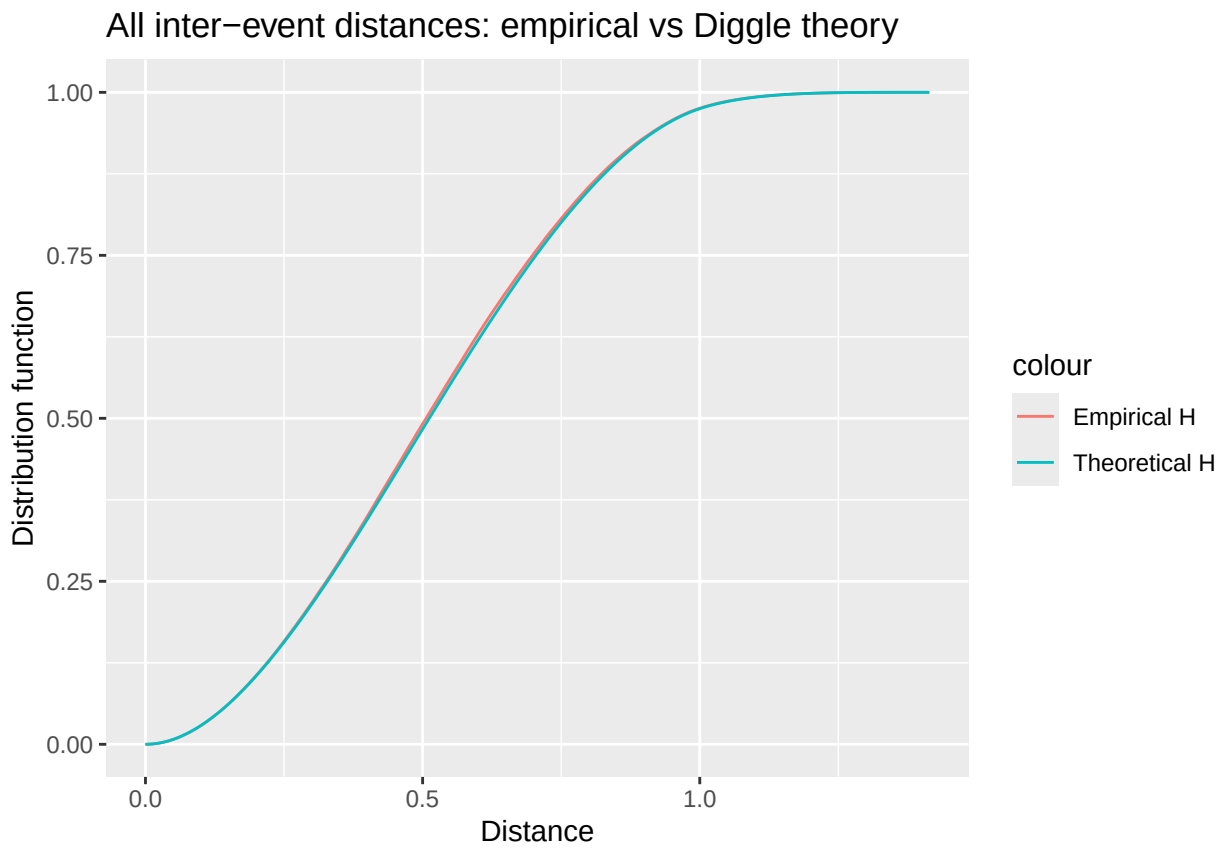
$$H(t) = \begin{cases} \pi t^2 - \frac{8}{3}t^3 + \frac{1}{2}t^4, & 0 \le t \le 1 \\ \frac{1}{3} - 2t^2 - \frac{1}{2}t^4 + \frac{4}{3}\sqrt{t^2-1}(2t^2+1) + 2t^2 \sin^{-1}(2t^{-2}-1), & 1 < t \le \sqrt{2} \end{cases}$$

```r
# Turn the dist object into a numeric vector
all_distances_vector <- as.numeric(all_distances)

# Define Diggle's H(t) for a unit square
H_t <- function(t) {
  result <- numeric(length(t))

  part1 <- (t >= 0) & (t <= 1)
  result[part1] <- pi * t[part1]^2 -
    (8 / 3) * t[part1]^3 +
    0.5 * t[part1]^4

  part2 <- (t > 1) & (t <= sqrt(2))
  result[part2] <- 1/3 -
    2 * t[part2]^2 -
    0.5 * t[part2]^4 +
    (4 / 3) * sqrt(t[part2]^2 - 1) * (2 * t[part2]^2 + 1) +
    2 * t[part2]^2 * asin(2 / t[part2]^2 - 1)

  result[t > sqrt(2)] <- 1
  result[t < 0] <- 0

  return(result)
}

# Create a grid of distances in my 1 x 1 square
t_grid <- seq(0, sqrt(2), length.out = 200)

# Empirical distribution function for all pairwise distances
empirical_H <- sapply(t_grid, function(t) {
  mean(all_distances_vector <= t)
```

```
})

theoretical_H <- H_t(t_grid)

# Put results into a data frame
all_distance_df <- data.frame(
  t = t_grid,
  empirical_H = empirical_H,
  theoretical_H = theoretical_H
)

head(all_distance_df)
```

```
##             t  empirical_H theoretical_H
## 1 0.000000000 0.0000000000  0.0000000000
## 2 0.007106601 0.0001401401  0.0001577065
## 3 0.014213202 0.0006026026  0.0006270128
## 4 0.021319802 0.0014074074  0.0014022224
## 5 0.028426403 0.0024084084  0.0024776691
## 6 0.035533004 0.0038138138  0.0038477176
```

```
ggplot(all_distance_df, aes(x = t)) +
  geom_line(aes(y = empirical_H, colour = "Empirical H")) +
  geom_line(aes(y = theoretical_H, colour = "Theoretical H")) +
  labs(
    title = "All inter-event distances: empirical vs Diggle theory",
    x = "Distance",
    y = "Distribution function",
  )
```



6 Get the nearest neighbor distance for each point

```
# Turn the points tibble into a matrix
point_matrix <- as.matrix(points)

# Compute the full distance matrix
distance_matrix <- as.matrix(dist(point_matrix))

# The distance from one point to itself is 0
# We change the diagonal to Inf so it will not be chosen as the nearest neighbor
diag(distance_matrix) <- Inf

# For each point, find the smallest distance
nearest_distances <- apply(distance_matrix, 1, min)

# Print the nearest neighbor distances
head(nearest_distances)
```

```
##           1           2           3           4           5           6
## 0.005072368 0.020520571 0.009527561 0.005913519 0.014358133 0.009303618
```

7 Compare the empirical nearest neighbor distribution with Diggle's theory

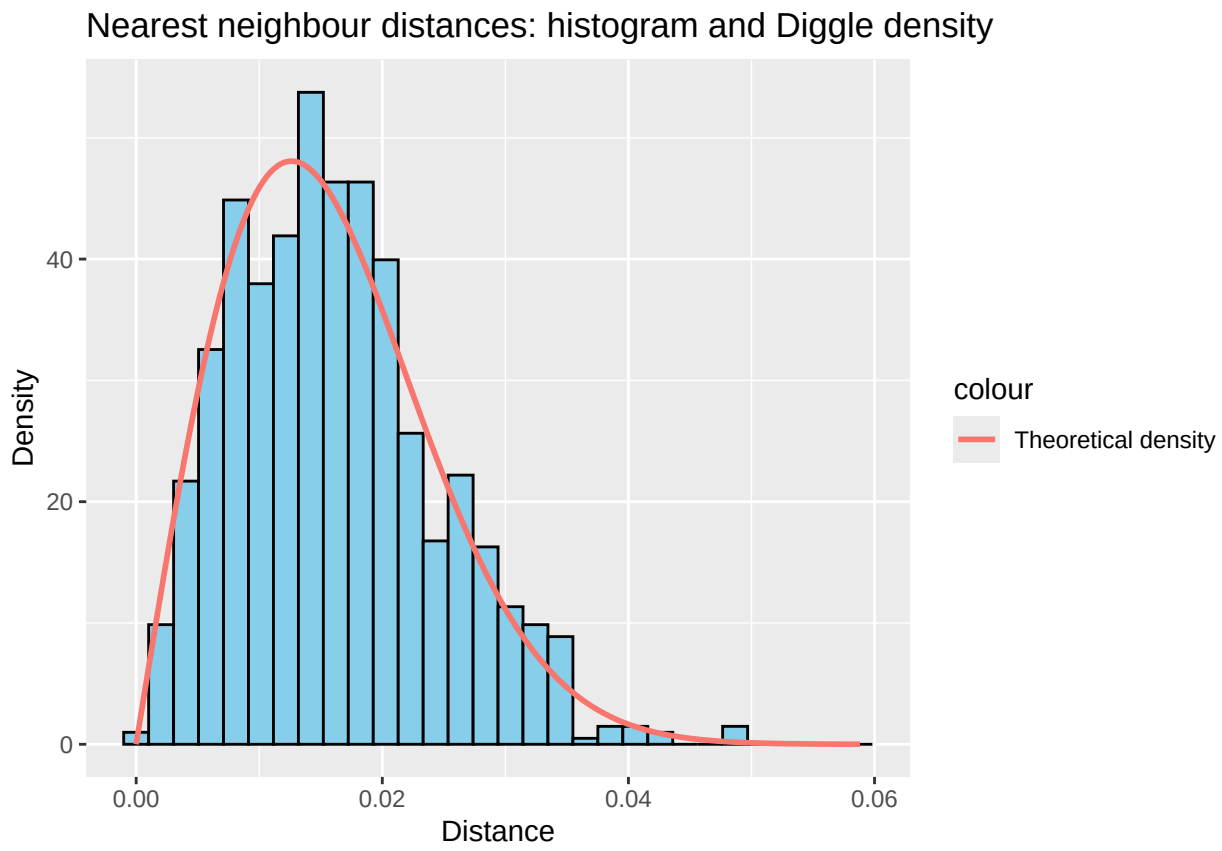
The nearest-neighbour density function:

$$g(y) = 2\pi\lambda y \exp(-\lambda\pi y^2)$$

```
# Intensity lambda = number of points / area
lambda <- n1 / (1 * 1)

g_y <- function(y, lambda) {
  2 * pi * lambda * y * exp(-lambda * pi * y^2)
}

# Create a grid of distance values
y_grid <- seq(0, max(nearest_distances) * 1.2, length.out = 200)

nn_density_df <- data.frame(
  y = y_grid,
  theoretical_g = g_y(y_grid, lambda)
)

head(nn_density_df)
```

```
##              y theoretical_g
## 1 0.0000000000      0.000000
## 2 0.0002955351      1.856392
## 3 0.0005910702      3.709729
## 4 0.0008866052      5.556965
## 5 0.0011821403      7.395069
## 6 0.0014776754      9.221037
```

```
ggplot(tibble(nn = nearest_distances), aes(x = nn)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 30,
    fill = "skyblue",
    colour = "black"
  ) +
  geom_line(
    data = nn_density_df,
    aes(x = y, y = theoretical_g, colour = "Theoretical density"),
    linewidth = 1
  ) +
  labs(
    title = "Nearest neighbour distances: histogram and Diggle density",
    x = "Distance",
    y = "Density",
  )
```



8 Add marker particles

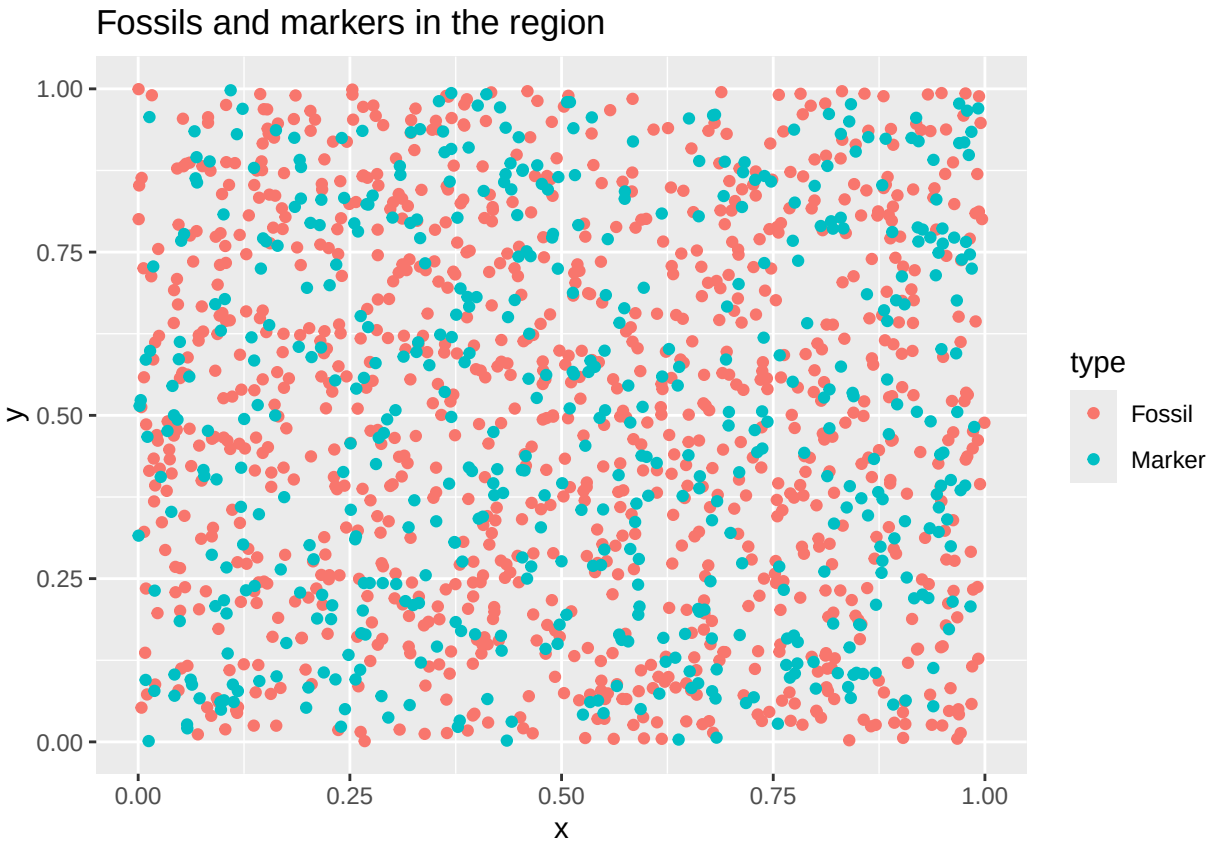
We now add marker particles into the same 1 x 1 square region.

```
# Number of marker particles
n_marker1 <- 500

# Generate marker coordinates
marker_x <- runif(n_marker1, min = 0, max = 1)
marker_y <- runif(n_marker1, min = 0, max = 1)

# Put marker coordinates into a tibble
markers <- tibble(x = marker_x, y = marker_y)

# Print the first few marker coordinates
head(markers)
```

```
## # A tibble: 6 x 2
##       x      y
##   <dbl>  <dbl>
## 1 0.242 0.413
## 2 0.480 0.378
## 3 0.840 0.392
## 4 0.992 0.970
```

```
## 5 0.244 0.0501
## 6 0.623 0.123
```

```r
# Add labels so we can plot fossils and markers together

points$type <- "Fossil"
markers$type <- "Marker"

all_points <- rbind(points, markers)

ggplot(all_points, aes(x = x, y = y, colour = type)) +
  geom_point() +
  ggtitle("Fossils and markers in the region")
```



9 Check all inter-event distances for marker particles

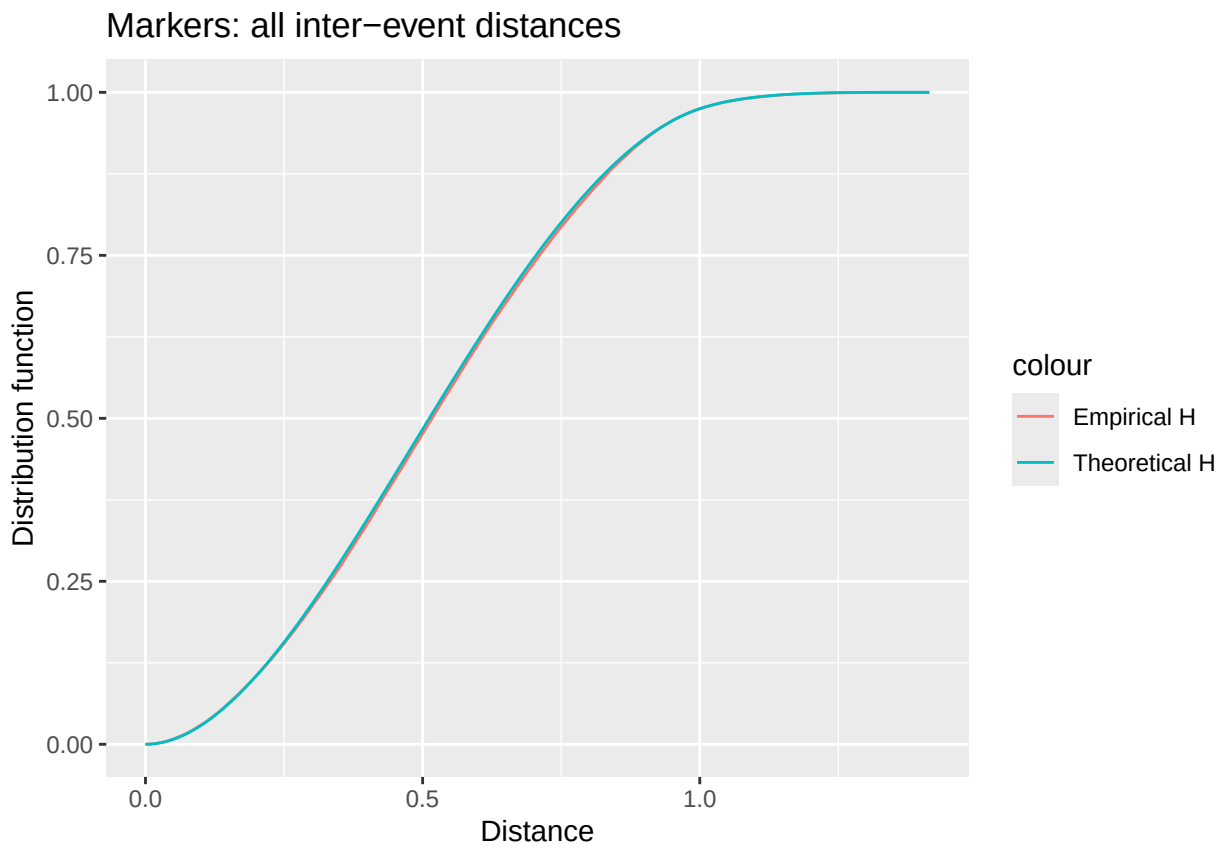
We now compare the empirical distribution of all marker-to-marker distances with Diggle's theoretical H(t).

```r
# Convert marker tibble to a matrix
marker_matrix <- as.matrix(markers[, c("x", "y")])

# Compute all pairwise distances between marker particles
marker_all_distances <- dist(marker_matrix)
```

```r
# Turn the dist object into a numeric vector
marker_ad_vector <- as.numeric(marker_all_distances)

# Use the same distance grid as before
marker_t_grid <- seq(0, sqrt(2), length.out = 200)

# Empirical distribution function for all marker distances
marker_empirical_H <- sapply(marker_t_grid, function(t) {
  mean(marker_ad_vector <= t)
})

# Theoretical H(t) for a unit square
marker_theoretical_H <- H_t(marker_t_grid)

# Put results into a data frame
marker_ad_df <- data.frame(
  t = marker_t_grid,
  empirical_H = marker_empirical_H,
  theoretical_H = marker_theoretical_H
)

head(marker_ad_df)
```

```
##             t  empirical_H theoretical_H
## 1 0.000000000 0.0000000000  0.0000000000
## 2 0.007106601 0.0002084168  0.0001577065
## 3 0.014213202 0.0007535070  0.0006270128
## 4 0.021319802 0.0016833667  0.0014022224
## 5 0.028426403 0.0027174349  0.0024776691
## 6 0.035533004 0.0041042084  0.0038477176
```

```r
ggplot(marker_ad_df, aes(x = t)) +
  geom_line(aes(y = empirical_H, colour = "Empirical H")) +
  geom_line(aes(y = theoretical_H, colour = "Theoretical H")) +
  labs(
    title = "Markers: all inter-event distances",
    x = "Distance",
    y = "Distribution function",
  )
```



10 Check nearest neighbor distances for marker particles

We now compute the nearest neighbor distance for each marker particle.

```
# Compute the full distance matrix for marker particles
marker_matrix1 <- as.matrix(dist(marker_matrix))

# Ignore the distance from each point to itself
diag(marker_matrix1) <- Inf

# For each marker particle, find the nearest neighbor distance
marker_nearest_distances <- apply(marker_matrix1, 1, min)

# Print the nearest neighbor distances
head(marker_nearest_distances)
```

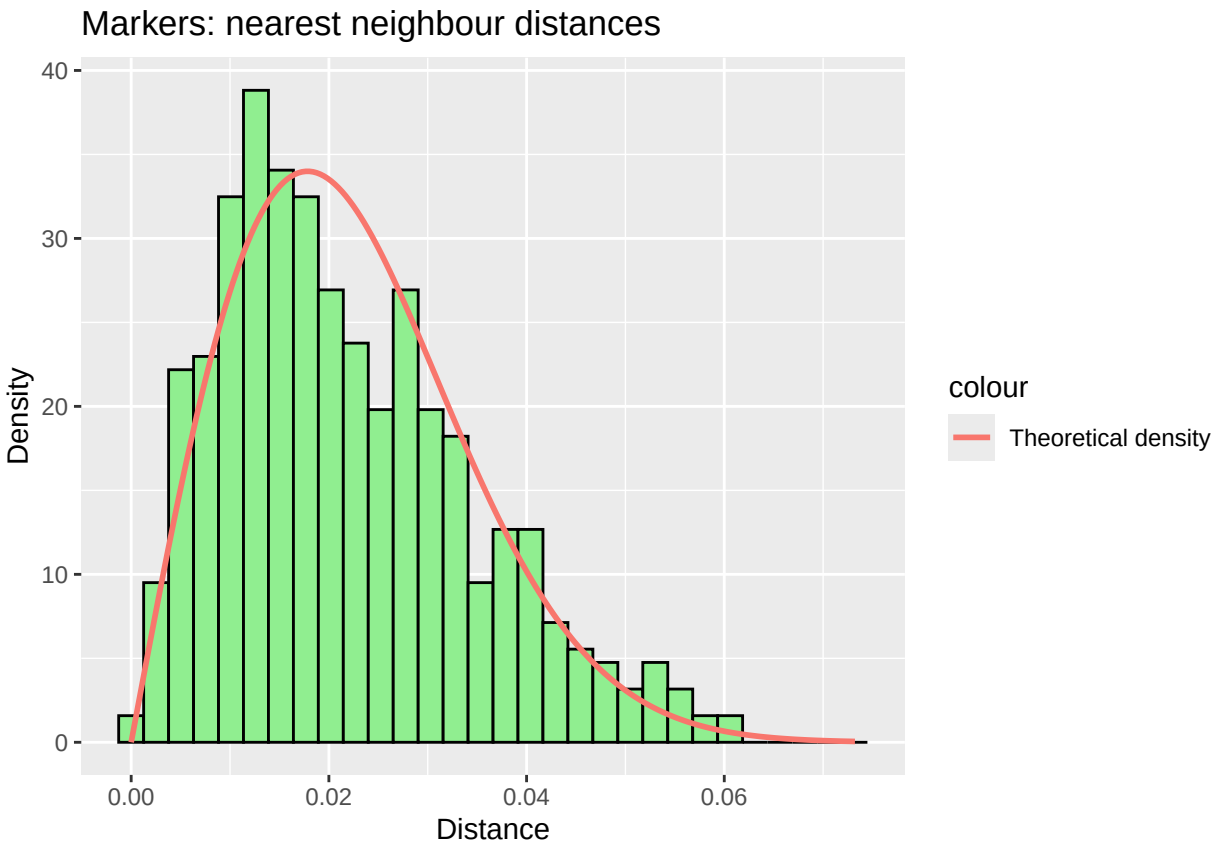
```
##          1          2          3          4          5          6
## 0.04051546 0.02772753 0.02300966 0.01353009 0.02726817 0.01292752
```

```
# Intensity lambda for marker particles
lambda_marker <- n_marker1 / (1 * 1)

# Create a grid of distance values
```

```r
marker_y_grid <- seq(0, max(marker_nearest_distances) * 1.2, length.out = 200)

marker_density_df <- data.frame(
  y = marker_y_grid,
  theoretical_g = g_y(marker_y_grid, lambda_marker)
)

head(marker_density_df)
```

```
##              y theoretical_g
## 1 0.0000000000      0.000000
## 2 0.0003679241      1.155622
## 3 0.0007358481      2.309770
## 4 0.0011037722      3.460973
## 5 0.0014716962      4.607767
## 6 0.0018396203      5.748697
```

```r
ggplot(tibble(nn = marker_nearest_distances), aes(x = nn)) +
  geom_histogram(aes(y = after_stat(density)),
                 bins = 30, fill = "lightgreen", colour = "black") +
  geom_line(data = marker_density_df,
            aes(x = y, y = theoretical_g, colour = "Theoretical density"),
            linewidth = 1) +
  labs(
    title = "Markers: nearest neighbour distances",
    x = "Distance",
    y = "Density",
  )
```



11 Set fields of view (FOVs)

We now place a number of square FOVs in the study region. Some FOVs are used for calibration, and some are used for extrapolation.

```
n_cal <- 20

n_ext <- 20

fov_size <- 0.1

region_size <- 1

# Check that the region can be evenly divided into square FOV cells
n_side <- region_size / fov_size
```

```
if (abs(n_side - round(n_side)) > 1e-10) {
  stop("fov_size must divide the region size exactly.")
}

n_side <- as.integer(round(n_side))

# Create all grid cell positions
all_grid <- expand.grid(
```

```
  x = seq(0, region_size - fov_size, by = fov_size),
  y = seq(0, region_size - fov_size, by = fov_size)
)

# Remove the outermost edge cells to reduce edge effects
inner_grid <- subset(
  all_grid,
  x > 0 & x < (region_size - fov_size) &
  y > 0 & y < (region_size - fov_size)
)

# Check that enough inner cells exist
if (nrow(inner_grid) < n_cal + n_ext) {
  stop("Not enough inner grid cells for the requested number of FOVs.")
}

# Calibration FOVs
cal_idx <- sample(1:nrow(inner_grid), size = n_cal, replace = FALSE)
cal_fovs <- inner_grid[cal_idx, ]

# Extrapolation FOVs from remaining inner cells
remaining_grid <- inner_grid[-cal_idx, ]
ext_idx <- sample(1:nrow(remaining_grid), size = n_ext, replace = FALSE)
ext_fovs <- remaining_grid[ext_idx, ]

cal_x <- cal_fovs$x
cal_y <- cal_fovs$y

ext_x <- ext_fovs$x
ext_y <- ext_fovs$y
```

12 Count fossils in calibration FOVs

In the calibration FOVs, we count the number of fossil particles.

```
cal_fossil <- numeric(n_cal)

for (i in 1:n_cal) {
  cal_fossil[i] <- sum(
    points$x >= cal_x[i] &
    points$x < cal_x[i] + fov_size &
    points$y >= cal_y[i] &
    points$y < cal_y[i] + fov_size
  )
}

cal_df <- tibble(
  fov = 1:n_cal,
  x = cal_x,
  y = cal_y,
  fossil = cal_fossil
)
```

```r
print(cal_df)
```

```
## # A tibble: 20 x 4
##      fov     x     y fossil
##    <int> <dbl> <dbl>  <dbl>
##  1     1   0.6   0.7     10
##  2     2   0.1   0.6      8
##  3     3   0.8   0.8     14
##  4     4   0.6   0.4     10
##  5     5   0.1   0.1      8
##  6     6   0.7   0.2     13
##  7     7   0.3   0.5     10
##  8     8   0.1   0.4     14
##  9     9   0.4   0.6      8
## 10    10   0.5   0.3     15
## 11    11   0.8   0.4     13
## 12    12   0.2   0.6     12
## 13    13   0.4   0.3     10
## 14    14   0.1   0.3      3
## 15    15   0.2   0.7      7
## 16    16   0.2   0.1      6
## 17    17   0.2   0.4      7
## 18    18   0.8   0.3      8
## 19    19   0.8   0.5      8
## 20    20   0.7   0.3     11
```

13 Count markers in extrapolation FOVs

In the extrapolation FOVs, we count the number of marker particles.

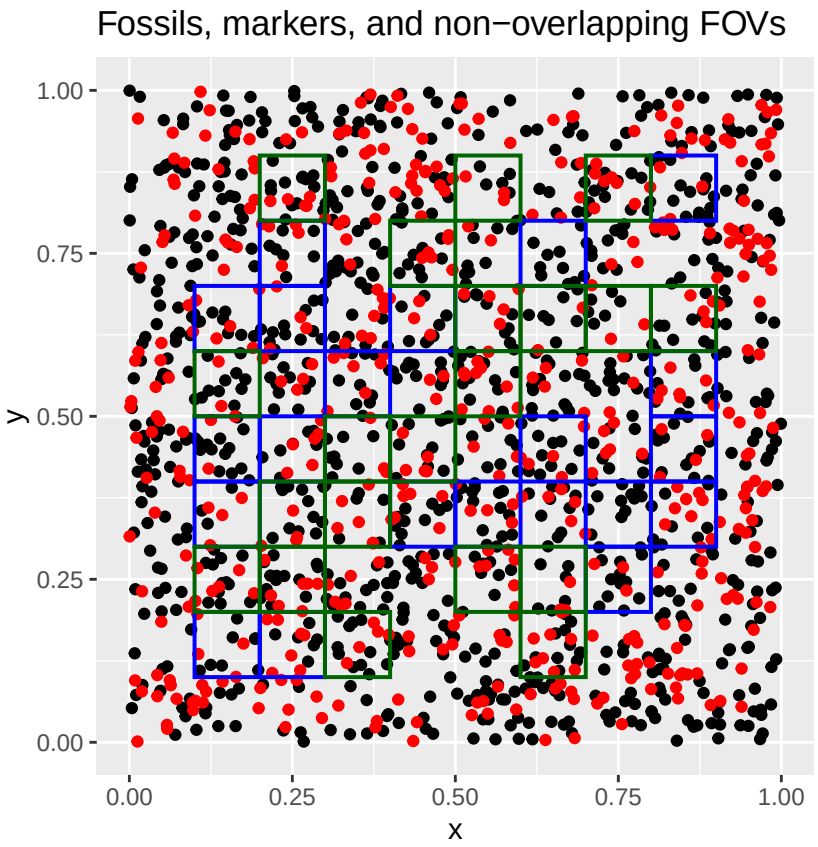
```r
ext_marker <- numeric(n_ext)

for (i in 1:n_ext) {
  ext_marker[i] <- sum(
    markers$x >= ext_x[i] &
    markers$x < ext_x[i] + fov_size &
    markers$y >= ext_y[i] &
    markers$y < ext_y[i] + fov_size
  )
}

ext_df <- tibble(
  fov = 1:n_ext,
  x = ext_x,
  y = ext_y,
  marker = ext_marker
)

print(ext_df)
```

```
## # A tibble: 20 x 4
```

```
##       fov     x     y marker
##     <int> <dbl> <dbl>  <dbl>
##  1     1   0.1   0.2      6
##  2     2   0.4   0.7      6
##  3     3   0.3   0.1      5
##  4     4   0.2   0.3      4
##  5     5   0.5   0.5     10
##  6     6   0.6   0.1      8
##  7     7   0.2   0.8      5
##  8     8   0.7   0.6      2
##  9     9   0.1   0.5      2
## 10    10   0.5   0.6      5
## 11    11   0.3   0.4      3
## 12    12   0.2   0.2      7
## 13    13   0.5   0.8      3
## 14    14   0.4   0.4      5
## 15    15   0.7   0.8      8
## 16    16   0.5   0.2      7
## 17    17   0.6   0.2      3
## 18    18   0.3   0.3      6
## 19    19   0.6   0.6      2
## 20    20   0.8   0.6      4
```

Show the non-overlapping FOVs

```
ggplot() +
  geom_point(data = points, aes(x = x, y = y), colour = "black") +
  geom_point(data = markers, aes(x = x, y = y), colour = "red") +
  geom_rect(
    data = cal_df,
    aes(xmin = x, xmax = x + fov_size, ymin = y, ymax = y + fov_size),
    fill = NA, colour = "blue", linewidth = 0.7
  ) +
  geom_rect(
    data = ext_df,
    aes(xmin = x, xmax = x + fov_size, ymin = y, ymax = y + fov_size),
    fill = NA, colour = "darkgreen", linewidth = 0.7
  ) +
  coord_fixed(xlim = c(0, 1), ylim = c(0, 1)) +
  ggtitle("Fossils, markers, and non-overlapping FOVs")
```



14 Estimate the number of fossils in the extrapolation-count FOVs

We use the mean fossil count from calibration FOVs($\bar{Y}_{3x}$) and multiply by the number of extrapolation FOVs(N_{3E}).

$$\hat{x} = \bar{Y}_{3x} \times N_{3E}$$

```
# Mean fossil count in calibration FOVs
ybar <- mean(cal_df$fossil)
print(ybar)
```

```
## [1] 9.75
```

```
# Estimated fossil count in extrapolation-count FOVs
xhat <- ybar * n_ext
print(xhat)
```

```
## [1] 195
```

15 Estimate concentration

We use the concentration formula

$$c_{Fx} = \frac{\hat{x} N_1 Y_1}{nV}$$

In this simulation:

- N_1Y_1 is the total number of marker particles added.
- n is the total number of markers counted in extrapolation FOVs.
- V is the study area, here taken as 1.

```
# Total number of markers counted in extrapolation FOVs
n_count <- sum(ext_df$marker)

# Total number of markers added
marker_total <- n_marker1

# Sample volume / area
V <- 1

# Estimated concentration
conc <- (xhat * marker_total) / (n_count * V)
print(conc)
```

```
## [1] 965.3465
```

16 Estimate error

The percentage error formula:

$$\sigma_{Fx} = 100\sqrt{\left(\frac{s_{1P}}{\sqrt{N_1}}\right)^2 + \left(\frac{s_{3P}}{\sqrt{N_{3C}}}\right)^2 + \left(\frac{\sqrt{n}}{n}\right)^2}$$

The total number of marker particles in this simulation is fixed and known exactly, the marker-dose error term is ignored here, so We use a simplified version of the percentage error formula:

$$\sigma_{Fx} = 100\sqrt{\left(\frac{s_{3P}}{\sqrt{N_{3C}}}\right)^2 + \left(\frac{\sqrt{n}}{n}\right)^2}$$

where:

- N_{3c} is the number of calibration FOVs.
- s_{3p} is the proportional sample standard deviation of fossil counts.
- n is the total counted markers in extrapolation FOVs

```
# Proportional standard deviation for fossil counts
s3p <- sd(cal_df$fossil) / mean(cal_df$fossil)

# Percentage error
err <- 100 * sqrt(
  (s3p / sqrt(n_cal))^2 +
  (sqrt(n_count) / n_count)^2
)
print(err)
```

```
## [1] 12.21978
```

17 Repeat the estimation many times

We now repeat the FOV procedure many times to get more stable results.

```
set.seed(456)
n <- 1000
n_marker <- 500

# Number of repeated simulations
n_rep <- 200

# Record start time
start_t <- Sys.time()

# True concentration
true_conc <- n / V

# Build fixed grid of candidate FOV locations
region_size <- 1
n_side <- region_size / fov_size

if (abs(n_side - round(n_side)) > 1e-10) {
  stop("fov_size must divide the region size exactly.")
}

n_side <- as.integer(round(n_side))

all_grid <- expand.grid(
  x = seq(0, region_size - fov_size, by = fov_size),
  y = seq(0, region_size - fov_size, by = fov_size)
)

# Remove outermost edge cells to reduce edge effects
inner_grid <- subset(
  all_grid,
  x > 0 & x < (region_size - fov_size) &
  y > 0 & y < (region_size - fov_size)
)

if (nrow(inner_grid) < n_cal + n_ext) {
  stop("Not enough inner grid cells for the requested number of FOVs.")
}

# Store results
conc_vec <- numeric(n_rep)
err_vec <- numeric(n_rep)
diff_vec <- numeric(n_rep)
percent_diff_vec <- numeric(n_rep)
effort_vec <- numeric(n_rep)

for (k in 1:n_rep) {

  # New fossil points
  x <- runif(n, min = 0, max = 1)
```

```r
y <- runif(n, min = 0, max = 1)
points_k <- tibble(x = x, y = y)

# New marker points
mx <- runif(n_marker, min = 0, max = 1)
my <- runif(n_marker, min = 0, max = 1)
markers_k <- tibble(x = mx, y = my)

# Select calibration FOVs from the fixed grid
cal_idx <- sample(1:nrow(inner_grid), size = n_cal, replace = FALSE)
cal_fovs <- inner_grid[cal_idx, ]

# Select extrapolation FOVs from the remaining grid cells
remaining_grid <- inner_grid[-cal_idx, ]
ext_idx <- sample(1:nrow(remaining_grid), size = n_ext, replace = FALSE)
ext_fovs <- remaining_grid[ext_idx, ]

cal_x <- cal_fovs$x
cal_y <- cal_fovs$y

ext_x <- ext_fovs$x
ext_y <- ext_fovs$y

# Count fossils in calibration FOVs
cal_fossil <- numeric(n_cal)

for (i in 1:n_cal) {
  cal_fossil[i] <- sum(
    points_k$x >= cal_x[i] &
    points_k$x <  cal_x[i] + fov_size &
    points_k$y >= cal_y[i] &
    points_k$y <  cal_y[i] + fov_size
  )
}

# Count markers in extrapolation FOVs
ext_marker <- numeric(n_ext)

for (i in 1:n_ext) {
  ext_marker[i] <- sum(
    markers_k$x >= ext_x[i] &
    markers_k$x <  ext_x[i] + fov_size &
    markers_k$y >= ext_y[i] &
    markers_k$y <  ext_y[i] + fov_size
  )
}

# Effort for this simulation
omega <- 2
x_total <- sum(cal_fossil)
n_total <- sum(ext_marker)
effort_vec[k] <- (omega * n_cal) + x_total + (omega * n_ext) + n_total
```

```r
  # Estimate concentration
  ybar <- mean(cal_fossil)
  xhat <- ybar * n_ext
  n_count <- sum(ext_marker)

  if (n_count == 0 || ybar == 0) {
    conc_vec[k] <- NA
    err_vec[k] <- NA
    diff_vec[k] <- NA
    percent_diff_vec[k] <- NA
  } else {
    conc_vec[k] <- (xhat * n_marker) / (n_count * V)

    s3p <- sd(cal_fossil) / mean(cal_fossil)
    err_vec[k] <- 100 * sqrt(
      (s3p / sqrt(n_cal))^2 +
      (sqrt(n_count) / n_count)^2
    )

    diff_vec[k] <- conc_vec[k] - true_conc
    percent_diff_vec[k] <- 100 * (conc_vec[k] - true_conc) / true_conc
  }
}

# Record total time
all_t <- Sys.time() - start_t
```

```r
cat("True concentration =", true_conc, "\n")
```

```
## True concentration = 1000
```

```r
cat("Mean estimated concentration =", mean(conc_vec, na.rm = TRUE), "\n")
```

```
## Mean estimated concentration = 1012.541
```

```r
cat("SD of estimated concentration =", sd(conc_vec, na.rm = TRUE), "\n")
```

```
## SD of estimated concentration = 114.2747
```

```r
cat("Mean error =", mean(err_vec, na.rm = TRUE), "\n")
```

```
## Mean error = 12.37873
```

```r
cat("SD of error =", sd(err_vec, na.rm = TRUE), "\n")
```

```
## SD of error = 0.8284224
```

```r
cat("Mean difference =", mean(diff_vec, na.rm = TRUE), "\n")
```

```
## Mean difference = 12.54071
```

```r
cat("SD of difference =", sd(diff_vec, na.rm = TRUE), "\n")
```

```
## SD of difference = 114.2747
```

```r
cat("Mean percent difference =", mean(percent_diff_vec, na.rm = TRUE), "\n")
```

```
## Mean percent difference = 1.254071
```

```r
cat("SD of percent difference =", sd(percent_diff_vec, na.rm = TRUE), "\n")
```

```
## SD of percent difference = 11.42747
```

```r
cat("Mean effort =", mean(effort_vec, na.rm = TRUE), "\n")
```

```
## Mean effort = 378.38
```

```r
cat("SD of effort =", sd(effort_vec, na.rm = TRUE), "\n")
```

```
## SD of effort = 15.3002
```

```r
cat("Total runtime =", all_t, "\n")
```

```
## Total runtime = 1.089234
```

18 Compare theoretical error with observed simulation variability

To assess the accuracy of the theoretical error estimate, we compare it with the observed variability in the concentration estimates across repeated simulations.

Their sample standard deviation is

$$SE_c = \sqrt{\frac{1}{R-1}\sum_{j=1}^{R}(\hat{c}_j - \bar{c})^2}$$

The simulation-based coefficient of variation is then

$$CV(c) = 100 \times \frac{SE_c}{\bar{c}}$$

```r
mean_conc <- mean(conc_vec, na.rm = TRUE)
sd_conc <- sd(conc_vec, na.rm = TRUE)

mean_err <- mean(err_vec, na.rm = TRUE)
sd_err <- sd(err_vec, na.rm = TRUE)

# Simulation-based coefficient of variation (%)
sim_cv <- 100 * sd_conc / mean_conc

# Difference between theoretical error and observed variability
err_diff <- mean_err - sim_cv
err_percent_diff <- 100 * err_diff / sim_cv

cat("Mean theoretical error =", mean_err, "\n")
```

```
## Mean theoretical error = 12.37873
```

```
cat("SD of theoretical error =", sd_err, "\n")
```

```
## SD of theoretical error = 0.8284224
```

```
cat("Simulation-based CV =", sim_cv, "\n")
```

```
## Simulation-based CV = 11.28594
```

```
cat("Difference between theoretical error and simulation CV =", err_diff, "\n")
```

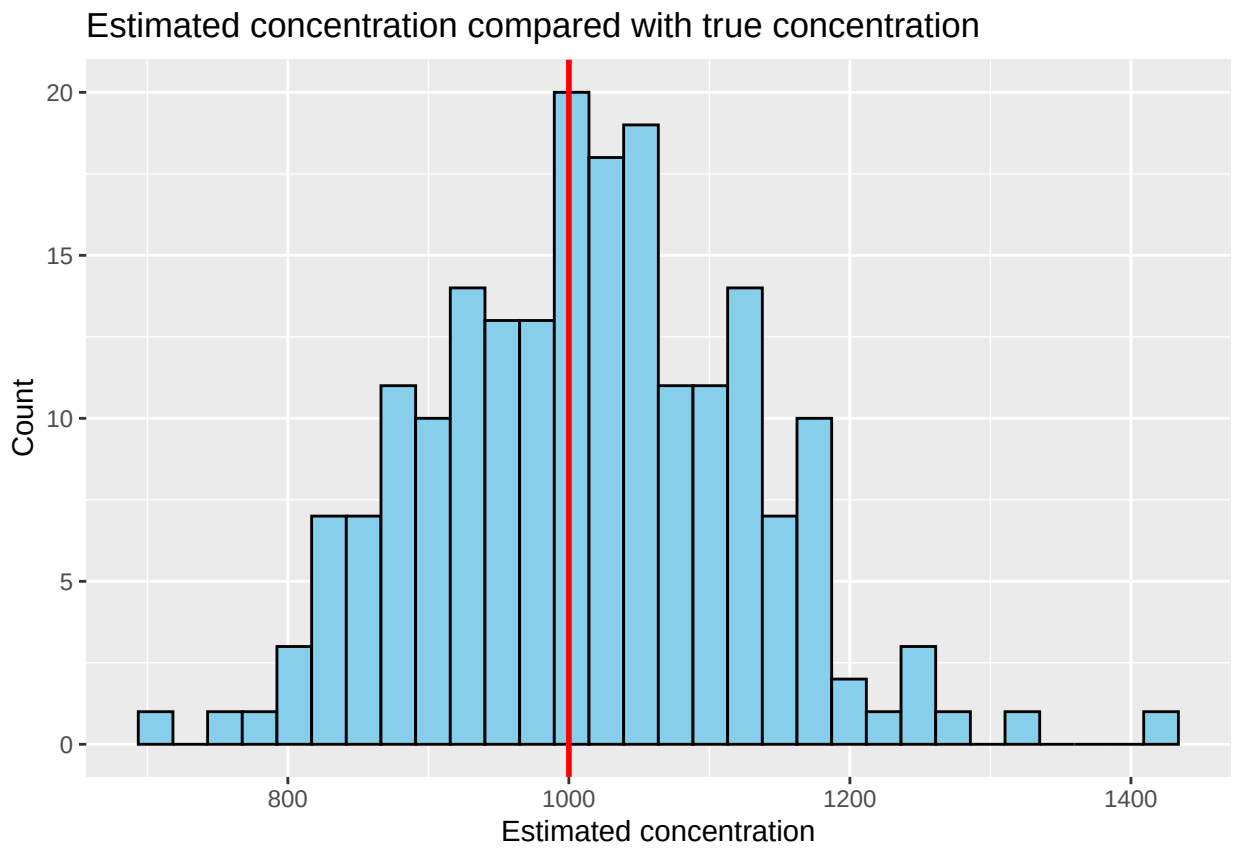
```
## Difference between theoretical error and simulation CV = 1.092789
```

```
cat("Percent difference =", err_percent_diff, "\n")
```

```
## Percent difference = 9.68275
```

19 Compare the estimated concentration to the true concentration

```
accuracy_df <- tibble(conc = conc_vec)

ggplot(accuracy_df, aes(x = conc)) +
  geom_histogram(bins = 30, fill = "skyblue", colour = "black") +
  geom_vline(xintercept = true_conc, colour = "red", linewidth = 1) +
  labs(
    title = "Estimated concentration compared with true concentration",
    x = "Estimated concentration",
    y = "Count"
  )
```




```
mean_conc <- mean(conc_vec, na.rm = TRUE)
mean_diff_true <- mean_conc - true_conc
percent_diff_true <- 100 * (mean_conc - true_conc) / true_conc

cat("True concentration =", true_conc, "\n")
```

```
## True concentration = 1000
```

```
cat("Mean estimated concentration =", mean_conc, "\n")
```

```
## Mean estimated concentration = 1012.541
```

Jensen's inequality

Jensen's inequality is a mathematical result about nonlinear functions and expected values. It says that, in general, the function of an expected value is not the same as the expected value of the function.

In simple terms, if a function is curved rather than linear, then "taking the average first" and "applying the function first" usually give different results.

The concentration formula:

$$c_{Fx} = \frac{\hat{x} N_1 Y_1}{nV}$$

The estimator is a ratio, and the denominator n is random. Because of Jensen's inequality, the average of the ratio is not exactly the same as the true concentration.

$$E\left(\frac{1}{n}\right) \neq \frac{1}{E(n)}$$

```
# Check bias
bias <- mean_conc - true_conc

# Correction factor
correction_factor <- true_conc / mean_conc

# Corrected estimates
conc_vec_corrected <- conc_vec * correction_factor

# Check corrected mean
mean_conc_corrected <- mean(conc_vec_corrected, na.rm = TRUE)

cat("Bias =", bias, "\n")
```

```
## Bias = 12.54071
```

```
cat("Correction factor =", correction_factor, "\n")
```

```
## Correction factor = 0.9876146
```

```
cat("Corrected mean concentration =", mean_conc_corrected, "\n")
```

```
## Corrected mean concentration = 1000
```

20 Calculate the effort required

The effort for the FOV subsampling formula is:

$$e_F = (\omega \times N_{3C}) + x + (\omega \times N_{3E}) + n$$

- e_F is the total effort required for the FOV subsampling method.
- ω is the effort needed to move from one FOV to another.(ω is 2, because the duration of a FOV transition is approximately equivalent to the time of two specimens.)
- N_{3C} is the number of calibration FOVs.
- x is the total number of fossils counted in the calibration FOVs.
- N_{3E} is the number of extrapolation FOVs.
- n is the total number of markers counted in the extrapolation FOVs.

In the repeated simulation above, the effort was calculated for each iteration.

```
cat("Mean effort =", mean(effort_vec, na.rm = TRUE), "\n")
```

```
## Mean effort = 378.38
```

```r
cat("SD of effort =", sd(effort_vec, na.rm = TRUE), "\n")
```

```
## SD of effort = 15.3002
```

```r
# Plot the effort
effort_df <- tibble(effort = effort_vec)

ggplot(effort_df, aes(x = effort)) +
  geom_histogram(bins = 30, fill = "lightgreen", colour = "black") +
  labs(
    title = "Distribution of effort across repeated simulations",
    x = "Effort",
    y = "Count"
  )
```